



HASHDIT

HashDit

Smart Contract Security

ListaDao AsterRewards Update

Security Audit Report

ListaDao

10 June 2026 – 15 June 2026

Disclaimer

This report is provided on an "as is" basis and does not constitute investment advice, a recommendation, or an endorsement of the audited project. HashDit makes no representations or warranties regarding the safety, functionality, or correctness of the smart contracts beyond the scope defined herein.

The audit was performed based on the source code provided at the time of the engagement. Any subsequent modifications to the codebase may invalidate the findings of this report. HashDit shall not be held liable for any losses or damages resulting from the use, deployment, or interaction with the audited contracts.

The scope of this audit is limited to the smart contracts listed in this report. It does not cover off-chain components, front-end interfaces, or third-party integrations unless explicitly stated. Users and developers should conduct their own due diligence before interacting with or deploying the audited contracts.

Audit Overview

Audit Period	10 June 2026 – 15 June 2026
Overall Risk [Optional]	Informational
Project Name	ListaDao
Github	https://github.com/lista-dao/lista-new-contracts/pull/35
Commit	feature/asterrewards-bot-notify

Smart Contract List

No.	Contract Name	Verdict	Details
1	AsterRewards.sol	Informational	[I01]
			[I02]
			[I04]
			[I05]
			[I06]
			[I07]
			[I08]
2	IAsterRewards.sol	Informational	[I03]

Findings

[I01] Missing validation of operator input in setOperator

Contract AsterRewards.sol

Severity Informational

Description

The AsterRewards.setOperator() function is missing two specific input validation during the updates of the operator address, with the current input validation checking only that the new value is not the zero address.

JavaScript

```
/* REWARD-SOURCE SETTER */
function setOperator(address newOperator) external onlyRole(MANAGER) {
    require(newOperator != address(0), "operator is zero");
    address oldOperator = operator;
    operator = newOperator;
    emit SetOperator(oldOperator, newOperator);
}
```

Description

Impact

1. It does not verify that the newOperator is different from the existing operator. As a result, a manager can set the operator to the same address that is already stored. This performs a redundant state write and emits a SetOperator event where the old and new operator values are identical, which can mislead off chain monitoring systems and event consumers into believing an operator change occurred when nothing actually changed.
2. If the operator and the feeReceiver are configured to be the same address, several issues arise. The fee becomes economically a no-op, as the fee never actually leaves the operator's control. It also produces misleading event data, because RewardsNotified(amount, fee, net) reports a non zero fee even though no fee was genuinely collected from a third party

Consider

Recommendation

- Adding a check that requires a newOperator != operator before updating the state
- Adding a validation check that prevents the operator and the feeReceiver from being set to the same address

Status	Open
--------	------

[I02] Operational risks introduced by moving notifyRewards to the BOT role with funds held in an external operator EOA

Contract	AsterRewards.sol
----------	------------------

Severity	Informational
----------	---------------

Description

The AsterRewards.notifyRewards() function has been changed so that it is now callable by the BOT role instead of the MANAGER role. The asterToken reward funds are no longer held in the previous MANAGER multisig wallet but now in a separate operator EOA, and notifyRewards() pulls those funds from the operator using safeTransferFrom. This design depends entirely on the ERC20 allowance that the operator EOA grants and on the security of that operator's private key.

Moreover, since PR #35 moved notifyRewards from onlyRole(MANAGER) pulling the caller's own funds to onlyRole(BOT) pulling from a third-party operator over its pre-granted ERC20 allowance, with no per-call, per-window, or cumulative cap. The only bound on amount is the operator's ERC20 approval to this contract; per the NatSpec the operator "must approve ASTER to this contract before BOT calls notifyRewards" with no stated per-call cap, and the operator is "the same entity/address as AsterVault.lisAsterManager" — a wallet holding live protocol principal, not a bounded reward tranche. Before this PR the puller and the funds-owner were the same principal (MANAGER spent its own balance), which implicitly bounded the pull; the PR breaks that coupling.

Description

JavaScript

```
function notifyRewards(uint256 amount) external override onlyRole(BOT)
whenNotPaused nonReentrant {
    require(amount > 0, "zero amount");
    require(operator != address(0), "operator not set");

    IERC20(asterToken).safeTransferFrom(operator, address(this),
amount);

    // Take fee only when both knobs are configured. Either feeRate=0 or
feeReceiver=0 means
    // no fee for this round -- MANAGER can stage the two settings in
any order without
    // bricking notifyRewards in between.
    uint256 fee = 0;
    if (feeRate > 0 && feeReceiver != address(0)) {
        fee = (amount * feeRate) / PRECISION;
        if (fee > 0) {
            IERC20(asterToken).safeTransfer(feeReceiver, fee);
        }
    }
}
```

```

    }
  }
  uint256 net = amount - fee;
  require(net > 0, "net is zero");

  emit RewardsNotified(amount, fee, net);
}

```

Two operational concerns follow from this:

1. First, the amount the BOT can pull at any given time is bounded only by the allowance set by the operator, so an overly large or unlimited allowance lets the BOT transfer more than the intended per round amount.
2. Second, because the reward funds sit in an externally owned account controlled by a single private key, a compromise of that key directly exposes the reward funds, as oppose to the multi-sig of the previous MANAGER role
3. Each call forwards feeRate ($\leq 30\%$) of the pulled amount to feeReceiver (AsterRewards.sol:127-130) and the BOT can distributeRewards the balance into LisAsterDistributor, permanently bumping totalNotified.

Consider the following:

Recommendation

- Limit the operator's allowance to only the amount expected to be needed for each round rather than granting a large or unlimited approval, and refresh it as needed so the BOT can never pull more than intended at any time.
- Manage the operator wallet under strong custody controls, for example a multisig or hardware backed signer, and define monitoring and rotation procedures so that a key compromise can be detected and contained quickly.

Status	Open
--------	------

[I03] SetOperator event fields are not indexed

Contract	IAsterRewards.sol
----------	-------------------

Severity	Informational
----------	---------------

Description	Description
	In the IAsterRewards and AsterRewards contracts, the SetOperator(address oldOperator, address newOperator) event emitted by setOperator does not mark either of its address parameters as indexed.

JavaScript

```
event SetOperator(address oldOperator, address newOperator);
```

Impact

Without indexed parameters, off chain indexers and tools cannot efficiently filter logs by operator address, making it harder to query historical operator changes. This is a minor issue with no impact on contract logic or funds.

Recommendation Consider adding the indexed keyword to the address parameters in the SetOperator event so that logs can be filtered by oldOperator and newOperator

Status Open

[I04] Possible stale manager allowance after notifyRewards funding source migrates to operator

Contract AsterRewards.sol

Severity Informational

Description

Description

In the updated version of the AsterRewards contract, the notifyRewards function pulls funds from the Operator, whereas the previous implementation pulled funds from the Manager. After the upgrade is applied, the Manager may still hold an unused token allowance previously granted to the AsterRewards contract for the old funding flow.

JavaScript

```
function notifyRewards(uint256 amount) external override
onlyRole(MANAGER) whenNotPaused nonReentrant {
    require(amount > 0, "zero amount");

    IERC20(asterToken).safeTransferFrom(msg.sender, address(this),
    amount);
```

Impact

Since this allowance is no longer needed under the new design, leaving it in place keeps an unnecessary approval active, which is poor hygiene and an avoidable source of risk.

Recommendation Revoke allowance of Manager for AsterReward contract

Status Open

[I05] No On-Chain Sync Constraint Between operator and IisAsterManager

Contract AsterRewards.sol, AsterVault.sol

Severity**Informational****Description**

The contract's NatSpec comment explicitly states that AsterRewards.operator and AsterVault.lisAsterManager must be the same address:

JavaScript

```
// AsterRewards - state variable declaration
/// @notice Lista-operated EOA on Astherus / Aster Chain; the ASTER
reward source pulled by
///          `notifyRewards`. Same entity/address as
`AsterVault.lisAsterManager`. It must
///          `approve` ASTER to this contract before BOT calls
`notifyRewards`.
address public operator;
```

Yet both are maintained by independent setters with no on-chain linkage:

JavaScript

```
// AsterVault - setLisAsterManager
function setLisAsterManager(address newManager) external
onlyRole(MANAGER) {
    lisAsterManager = newManager;
}
// AsterRewards - setOperator
function setOperator(address newOperator) external onlyRole(MANAGER) {
    require(newOperator != address(0), "operator is zero");
    operator = newOperator;
}
```

Description

The deployment script further confirms the disconnect: ASTER_VAULT_PROXY is already referenced but its lisAsterManager() is never read to initialize operator, leaving the two configuration lines entirely separate:

JavaScript

```
// deploy_lisaster_rewards_v2_bsc.s.sol - run()
address constant ASTER_VAULT_PROXY =
0xb3Df1b695D720dDc5906005DD5448DB160687C42;
// ...
rewards.setOperator(...); // ASTER_VAULT_PROXY.lisAsterManager() is
never read
```

Impact

An EOA rotation requires two separate MANAGER governance transactions across two contracts. If either is missed, reward distribution stops immediately or degrades over time: if setLisAsterManager is updated but setOperator is not, new yield flows to the

new EOA while the BOT continues pulling from the old address, causing notifyRewards to revert once the old address is exhausted; the reverse causes immediate failure. No funds are at risk, but users stop receiving rewards until a corrective governance transaction is submitted.

Derive operator directly from AsterVault in the deployment script to translate the documented constraint into an engineering-level guarantee:

Recommendation

JavaScript

```
address operatorAddr = IAsterVault(ASTER_VAULT_PROXY).lisAsterManager();
rewards.setOperator(operatorAddr);
require(rewards.operator() == operatorAddr, "operator not wired");
```

The EOA rotation runbook should require both contract setters to be called within the same multisig batch.

Status	Open
--------	------

[I06] Fee bypass via direct ASTER transfer to Rewards

Contract	AsterRewards.sol
----------	------------------

Severity	Informational
----------	---------------

Description

Description

Fees are charged only in notifyRewards(). distributeRewards() checks the contract's ASTER balance only and does not verify that funds entered via notifyRewards().

An alternate path exists: the operator transfers ASTER directly to Rewards, then the BOT calls distributeRewards(), skipping the fee. This differs from [I01] (operator == feeReceiver self-transfer fee), which still runs through notifyRewards().

Related Code

JavaScript

```
function notifyRewards(uint256 amount) external override onlyRole(BOT)
whenNotPaused nonReentrant {
    require(operator != address(0), "operator not set");
    IERC20(asterToken).safeTransferFrom(operator, address(this),
amount);
    if (feeRate > 0 && feeReceiver != address(0)) {
        fee = (amount * feeRate) / PRECISION;
        IERC20(asterToken).safeTransfer(feeReceiver, fee);
    }
    // ...
}

function distributeRewards(uint256 amount) external override
onlyRole(BOT) whenNotPaused nonReentrant {
```

```

        require(amount <= IERC20(asterToken).balanceOf(address(this)),
            "exceeds balance");
        ILisAsterDistributor(distributor).notifyRewards(amount);
    }

```

Bypass path: transfer(rewards, amount) → distributeRewards(amount) (without notifyRewards).

Impact

Requires cooperation from the operator and/or BOT. No external theft path, but protocol fee revenue (e.g. 10%) can be avoided.

Recommendation

Enforce the notifyRewards funding path in the operator/BOT runbook. If on-chain enforcement is required, track net inflows only in notifyRewards and restrict distributeRewards to that balance.

Status

Open

[I07] Missing setOperator wiring in AsterRewards upgrade and deployment scripts

Contract

AsterRewards.sol

Severity

Informational

Description

Description

This PR is a UUPS upgrade that appends an operator storage slot. After upgrade the slot is zero and initialize does not set it. Until MANAGER calls setOperator(), notifyRewards() reverts with "operator not set".

The repo has an upgrade script for LisAsterDistributor but none for AsterRewards; deploy_lisaster_rewards_v2_bsc.s.sol also omits setOperator (the test setUp does call it).

Impact

Fail-closed — no unauthorized transfers — but reward ingestion halts after upgrade/deploy until MANAGER configures operator and the operator grants allowance.

Recommendation

Add an upgrade script: upgradeToAndCall → setOperator(IAsterVault(...).lisAsterManager()) → verify approve → unpause. Include setOperator in the greenfield deploy script as well.

Status

Open

[I08] emergencyWithdraw is a broad escape hatch on the routine MANAGER role, sending funds to the caller with no timelock or fixed sink

Contract

AsterRewards.sol

Severity

Informational

Description

emergencyWithdraw is gated by the same MANAGER role used for routine configuration (setFee*, setOperator, unpause), transfers an unbounded amount of any token to msg.sender (a caller-controlled destination, not a fixed treasury), and has no timelock. Because the contract holds undistributed staker rewards between notifyRewards and distributeRewards (pendingAster() == balanceOf(this)), a compromised MANAGER key can divert in-flight rewards owed to stakers to an arbitrary address.

Description

JavaScript

```
//AsterRewards.sol:116-120
function emergencyWithdraw(address token, uint256 amount) external
onlyRole(MANAGER) {
    require(token != address(0), "zero token");
    require(amount > 0, "zero amount");
    IERC20(token).safeTransfer(msg.sender, amount);
    emit EmergencyWithdrawn(token, msg.sender, amount);
}
```

Impact:

requires MANAGER (multisig) compromise, so this is a centralization / defense-in-depth hardening item rather than an unprivileged exploit. There is no local accounting desync (the contract keeps no cumulative state, and LisAsterDistributor.totalNotified is only bumped at distributeRewards, after the ASTER has physically left this contract — so withdrawing pre-distribute ASTER cannot desync the distributor).

Recommendation

Route the escape hatch through DEFAULT_ADMIN_ROLE behind the Timelock (as with _authorizeUpgrade), and/or send to a fixed treasury address rather than msg.sender. Consider excluding asterToken up to pendingAster() owed, or requiring paused() first.

Status

Open